



Immutable State



Learning Objectives

- Explain how a component can hold immutable state and then update it by copying its state before performing the update
- Build a React component that holds an immutable array as state and updates it



Time for some live coding

Let's try and push an item on to an array held in React state...

- <https://github.com/boolean-uk/react-immutable-state>



Arrays and State

React relies on **object identity** to understand what has changed during an update before rendering. It's entirely possible to mutate state, but React is opinionated – it encourages us to deal with changes **immutably**.

This means **whenever we change our array we should call the set state function with a copy of the array, and never modify the original array.**

This can feel a bit unusual at first but there are common techniques we can apply whenever we update an array or its elements:

- Add items to an array
- Remove items from an array
- Update items in an array



Intro

Let's use the [Todo app](#) example again

OPTIONS

Show completed ☒

ADD ITEM

Add

TODO

☐ Go shopping

☐ Work out

COMPLETED

☒ See the doctor



Adding Items

✗ Mutating todos array

```
todos.push({ text, completed: false })
```

✓ Creating a new array - long version

```
const newTodos = [...todos]
newTodos.push({ text, completed: false })
setTodos(newTodos)
```

✓ Creating a new array - short version

```
setTodos([...todos, { text, completed: false }])
```

Creates a **new array**, taking all existing elements from the todos array (`...todos`). The `...` is referred to as *spread syntax*.

See [MDN Documentation](#)

Creates a **new array** by copying todos and adding a new item.

See [MDN Documentation](#)

Rather than update the existing array, we make a *copy* of the array, add the new item to it, then call the set state (`setTodos`) function.



Removing Items

```
function removeTodo(target)
  const todosWithoutRemoved = todos.filter(function(todo) {
    return todo !== target
  })
  setTodos(todosWithoutRemoved)
}

{incompleteTodos.map(todo => (
  <li className="todo">
    <button className="delete" onClick={ () => removeTodo(todo) }>🗑️</button>
  </li>
)) }
```

Q: How is this **removeTodo** function working?



Removing Items

```
function removeTodo(target) {
  const todosWithoutRemoved = todos.filter(function(todo) {
    return todo !== target
  })
  setTodos(todosWithoutRemoved)
}

{incompleteTodos.map(todo => (
  <li className="todo">
    <button className="delete" onClick={() => removeTodo(todo)}>🗑️</button>
  </li>
))}
```

The trick here is, each todo item has its own onClick, and passes its own todo object to **removeTodo** when clicked.

We are relying on [referential equality](#) here to filter out the todo we pass as an **argument**.



Removing Items – Shortened

```
const removeTodo = target => { setTodos(todos.filter(todo => todo !== target)) }

{incompleteTodos.map(todo => (
  <li className="todo">
    <button className="delete" onClick={() => removeTodo(todo)}>🗑️</button>
  </li>
))}
```

We can also make use of **arrow functions** to shorten this code.



Removing Items

```
const initialTodos = [  
  { id: 1, text: 'Go shopping', completed: false },  
  { id: 2, text: 'Work out', completed: false },  
  { id: 3, text: 'See the doctor', completed: true }  
]  
  
const removeTodo = target => { setTodos(todos.filter(todo => todo !== target)) }
```

Q: What if our todos had IDs? Could you think of a different way to write this function?



Removing Items

```
const initialTodos = [  
  { id: 1, text: 'Go shopping', completed: false },  
  { id: 2, text: 'Work out', completed: false },  
  { id: 3, text: 'See the doctor', completed: true }  
]  
  
const removeTodo = target => { setTodos(todos.filter(todo => todo.id !== target.id)) }
```

A: We can filter an element out by its **id** instead of its reference!

Q: Is that better? Why?



Updating Items

```
function toggleTodoCompletion(target) {  
  const updatedTodos = todos.map(function(todo) {  
    if (todo === target) {  
      return { ...todo, completed: !todo.completed }  
    }  
    return todo  
  })  
  setTodos(updatedTodos)  
}
```

Q: How is this **toggleTodoCompletion** function working?



Updating Items

```
function toggleTodoCompletion(target) {  
  const updatedTodos = todos.map(function(todo) {  
    if (todo === target) {  
      return { ...todo, completed: !todo.completed }  
    }  
    return todo  
  })  
  setTodos(updatedTodos)  
}
```

Creates a **new object**, taking all existing properties from the todo object (`...todo`) then updating the completed property.

This is the *spread syntax* again -we can use it with Objects too!

See [MDN Documentation](#)

We map through all the existing todos. If the todo matches the one we are trying to remove, then we **create a new todo object**, copying all properties and then updating the completed property. For all other todos we return the existing object.

This results in a copy of the todos array, with our target todo update, that we can then pass to setTodos to update the state.



Updating Items – Shortened

```
const toggleTodoCompletion = (target) => {  
  const updatedTodos = todos.map(todo =>  
    todo === target ? { ...todo, completed: !todo.completed } : todo  
  )  
  setTodos(updatedTodos)  
}
```

As before, we can use arrow functions to shorten this code. This example also uses the ternary operator to shorten the if statement.



Time for some practice

- <https://github.com/boolean-uk/react-immutable-state>